

Çok Katmanlı Core–Mobile–Web Mimarisi: Core İş Mantığının Paylaşımı ve Evrimsel Yol Haritası

Ömer Faruk Yavuz

December 2025

Giriş

Güncel yazılım projelerinde, aynı domain mantığının birden fazla istemci tarafından (mobil, web, hatta zaman zaman sunucu tarafı süreçler) ortak olarak kullanılması giderek daha önemli hale gelmektedir. Özellikle modern JavaScript / TypeScript ekosisteminde, React Native ile geliştirilen mobil uygulamalar ve React / Next.js / Vite tabanlı web uygulamaları sıklıkla aynı iş kuralları, aynı veri modelleri ve benzer doğrulama mantıkları etrafında şekillenmektedir.

Bu bağlamda, *core* veya *business* olarak adlandırılan merkezi bir katmanda domain modellerinin, servislerin ve iş mantığının toplanması ve bu katmanın hem mobil hem de web istemcileri tarafından tüketilmesi, tekrar eden kodu azaltan, bakım maliyetini düşüren ve tutarlılığı artıran bir mimari yaklaşım sunar. Bununla birlikte, core katmanının hangi sorumlulukları üstlenmesi gerektiği, istemci projelere nasıl bağlanacağı ve zaman içinde domain odaklı bir yapıya (örneğin DDD benzeri bir organizasyona) nasıl evrilebileceği pratikte cevaplanması gereken sorulardır.

Bu dokümanda, öncelikle core–mobile–web üçlüsünün genel problem tanımını ve tipik dizin yapısı ele alınmakta, ardından core katmanının sınırları ve örnek business fonksiyonları tartışılmakta, son olarak da core paketinin farklı projeler arasında nasıl paylaştırılabileceği ve mimari yapının fazlar halinde nasıl evrilebileceği açıklanmaktadır.

Genel Problem Tanımı

Modern istemci tarafı uygulamalarda, aynı iş kurallarını ve domain mantığını birden fazla istemci türünde (örneğin mobil uygulama, web uygulaması, hatta zaman zaman sunucu tarafı Node.js süreçleri) yeniden kullanma ihtiyacı sıklıkla ortaya çıkar. Tipik durum şu şekildedir:

- Elinizde **ortak iş mantığını** barındıran bir katman bulunmaktadır: bu katman çoğunlukla “core” veya “business” projesi olarak adlandırılır.
- Bu core katmanındaki işlevlerin:
 - Mobil istemci (React Native),
 - Web istemci (React / Next.js / Vite),
 - Gerekirse Node.js tabanlı arka plan süreçleri

tarafından tüketilmesi hedeflenir.

Bu bağlamda yaygın mimari desen, üç ayrı proje veya paket üzerinden şu yapıdadır:

1. **Core / Business projesi:** Domain modelleri, servisler, API istemcileri, doğrulama (validation) mantığı ve UI'dan bağımsız hook'lar bu projede konumlandırılır.
2. **Mobile projesi:** React Native kullanarak iOS ve Android istemcilerini üreten proje.
3. **Web projesi:** React, Next.js veya Vite tabanlı web istemcisini üreten proje.

Hem mobil hem web uygulaması, mümkün olduğunca çok mantığı bu core projeden “tüketir”; böylece iş kuralları tek yerde tanımlanır ve tekrar kullanım maksimize edilir.

Üçlü Yapının Tipik Dizini ve Paket Yapısı

Bu mimari, hem tek bir monorepo içerisinde hem de fiziksel olarak ayrılmış projeler şeklinde kurgulanabilir. Klasik bir monorepo yapısı aşağıdaki gibi özetlenebilir:

```
my-app/
  package.json          # workspace tanımı
  tsconfig.base.json

packages/
  core/                 # business / domain / shared logic
    package.json
    src/
      index.ts
      domain/
      api/
      validation/

apps/
  mobile/               # React Native uygulaması
    package.json
    App.tsx
    ...
  web/                  # Web (React / Next / Vite) uygulaması
    package.json
    src/App.tsx
    ...
```

Bu yapıda iki temel ilke öne çıkar:

- **Core paketi**, sıradan bir TypeScript kütüphanesi gibi tasarlanır; içerisinde:

- `react-native` import'ları,
- DOM'a özgü `document`, `window` vb. kullanımlar

bulunmaz. Core katmanı yalnızca saf TypeScript/JavaScript ile ifade edilen modelleri, servisleri, API istemcilerini, doğrulama şemalarını ve UI'dan bağımsız hook'ları barındırır.

- **Mobile** ve **web** projeleri ise bu core paketini normal bir bağımlılık gibi kullanır; örneğin:

```
import { useAppointments } from '@org/core';
```

şeklinde işlevleri içe aktarır ve kendi UI katmanında tüketir.

Core Katmanının Sorumlulukları ve Sınırları

Core katmanında bulunması uygun olan unsurlar şunlardır:

- **Tipler ve modeller:** Örneğin `User`, `Appointment`, `Question`, `Medication` gibi domain nesneleri.
- **API istemcileri:** Örneğin `appointmentApi.getAppointments`, `authApi.login` gibi fonksiyonlar.
- **Doğrulama / şema tanımları:** `Zod`, `Yup` vb. kütüphanelerle tanımlanmış validation şemaları.
- **UI'dan bağımsız hook'lar:** Örneğin:

```
useAppointments() // sadece veri çeker ve state döndürür,  
                  // hangi komponentin bu veriyi göstereceğini bilmez.
```

Core katmanında *bulunmaması gereken* unsurlar ise şunlardır:

- React Native'e özgü import'lar (`View`, `Text`, `Platform` vb.),
- Web DOM API'leri (`document`, `window` vb.),
- Native modül bağımlılıkları (`react-native-keychain`, `react-native-device-info` vb.).

Bu sayede core katmanı, hem mobil hem web hem de gerekiyorsa Node.js sürüşleri tarafından tüketilebilen *ortak iş mantığı* katmanı haline gelir.

Mobil ve Web Projelerinin Core'dan Beslenmesi

Core katmanından sağlanan iş mantığı, mobil ve web projelerinde farklı UI bileşenleri ile birleştirilir. Örneğin, `useAppointments` adlı ortak bir hook hem React Native hem de web tarafında şu şekilde kullanılabilir:

Mobil (React Native) örneği:

```
// apps/mobile/App.tsx  
import React from 'react';  
import { View, Text } from 'react-native';  
import { useAppointments } from '@org/core/appointments';  
  
export default function App() {  
  const { appointments, isLoading } = useAppointments();  
  
  if (isLoading) return <Text>Yükleniyor...</Text>;  
  
  return (  

```

```

    <View>
      {appointments.map((a) => (
        <Text key={a.id}>{a.title}</Text>
      ))}
    </View>
  );
}

```

Web (React) örneği:

```

// apps/web/src/App.tsx
import React from 'react';
import { useAppointments } from '@org/core/appointments';

export default function App() {
  const { appointments, isLoading } = useAppointments();

  if (isLoading) return <p>Yükleniyor...</p>;

  return (
    <div>
      {appointments.map((a) => (
        <div key={a.id}>{a.title}</div>
      ))}
    </div>
  );
}

```

Görüldüğü üzere, *aynı* iş mantığı hook'u her iki tarafta da kullanılmakta; yalnızca UI katmanı ilgili platformun bileşenleri ile farklılaştırılmaktadır.

Core Paketin Farklı Projeler ve Makineler Arasında Paylaşımı

Yerel (Local) file: Bağımlılığı ile Kullanım

Tek bir geliştiricinin aynı makinede üç ayrı proje ile çalıştığı durumda, core paketi harici bir registry'e publish edilmeden de kullanılabilir. Bu senaryoda, dizin yapısı kabaca aşağıdaki gibi olabilir:

```
D:\Projects\  
  core-business\  
    mobile-app\  
    web-app\  

```

Core projesi, sıradan bir TypeScript kütüphanesi olarak yapılandırılır:

```
// core-business/package.json  
{  
  "name": "@org/core-business",  
  "version": "0.0.1",  
  "main": "dist/index.js",  
  "types": "dist/index.d.ts",  
  "scripts": {  
    "build": "tsc"  
  }  
}
```

Mobil ve web projelerinde ise core paketi, yerel dosya bağımlılığı şeklinde tanımlanır:

```
// mobile-app/package.json  
{  
  "dependencies": {  
    "@org/core-business": "file:../core-business"  
  }  
}  
  
// web-app/package.json  
{  
  "dependencies": {  
    "@org/core-business": "file:../core-business"  
  }  
}
```

Bu yaklaşımda:

- core-business projesi derlenir (npm run build),

- Diğer projelerde `npm install` veya benzeri komutlar çalıştırılarak yerel paket linklenir,
- Kod içinde normal bir paket gibi:

```
import { useCounter } from '@org/core-business';
```

ifadesiyle kullanılabilir.

Bu yöntem, **aynı makinede** ve **öngörülebilir klasör yapısında** çalışan tek geliştirici için pratik ve yeterli bir çözümdür; ancak farklı geliştiricilerin kendi izin yapıları olduğunda sürdürülebilir olmaktan çıkar.

Çok Geliştiricili Ortamda Local file: Yönteminin Sınırları

Birden fazla geliştiricinin, farklı klasör yapıları ile çalıştığı ekip ortamlarında:

- Geliştirici A için:

```
C:\Users\A\Projects\core-business  
C:\Users\A\Projects\mobile-app
```

- Geliştirici B için:

```
D:\Work\core-business  
D:\Work\mobile-app
```

gibi farklı dizinler söz konusu olabilir. Bu durumda "`file:../core-business`" biçimindeki görelî path tanımı, tüm makinelerde aynı anlamı taşımayacaktır. Bu nedenle, proje ve geliştirici sayısı arttıkça, core paketinin **merkezi bir kaynaktan** alınması daha doğru bir yaklaşım haline gelir.

Private Registry Üzerinden Paket Yayını

Çok geliştiricili ve uzun soluklu projelerde en sürdürülebilir çözüm, core paketini bir package registry (npm, GitHub Packages, GitLab Packages, kurumsal registry vb.) üzerinden yayımlamaktır.

Bu senaryoda:

- Core projesi, kendi `package.json` dosyasıyla bağımsız bir kütüphane olarak yapılandırılır.
- Paket, private veya public bir registry'e belirli bir sürüm numarasıyla (0.1.0, 0.2.0 vb.) publish edilir.
- Mobil ve web projeleri, `package.json` dosyalarında:

```
"dependencies": {  
  "@org/core-business": "0.1.0"  
}
```

biçiminde bu paketi referans alır.

Bu yaklaşımın başlıca avantajları şunlardır:

- Tüm geliştiriciler ve CI/CD süreçleri, core paketini aynı merkezi kaynaktan çeker.
- Sürümleme disiplinine uygun olarak, değişiklikler kontrollü biçimde yayılır.
- Makine veya klasör yapısına bağımlılık ortadan kalkar.

Git Üzerinden Bağımlılık Tanımı

Registry yönetmek istemeyen veya hızlı bir prototip aşamasında olan projelerde, core paketine doğrudan Git deposu üzerinden de bağımlılık tanımlanabilir:

```
"dependencies": {  
  "@org/core-business": "git+https://github.com/organization/core-business.git"  
}
```

veya SSH üzerinden:

```
"dependencies": {  
  "@org/core-business": "git+ssh://git@github.com:organization/core-business.git"  
}
```

Bu yöntemde:

- `npm install` veya `yarn install` komutu, ilgili Git deposunu indirir.
- Core paketi yine normal bir kütüphane gibi import edilir.

Buna karşılık:

- Sürümleme ve değişiklik yönetimi, registry tabanlı çözümlere göre daha dağınık olabilir.
- Belirli commit hash'leri veya branch'lerle çalışmak gerekebilir.

Monorepo ve Workspace Yaklaşımı

Bir diğer yaygın yaklaşım da, core, mobil ve web projelerinin **tek bir monorepo** içinde tutulmasıdır. Bu durumda dizin yapısı örneğin şu şekilde olabilir:

```
my-app/  
  package.json  
  packages/  
    core-business/  
  apps/  
    mobile/  
    web/
```

Tüm geliştiriciler aynı Git deposunu klonlar:

```
git clone git@github.com:organization/my-app.git
```

Bu durumda klasör hiyerarşisi herkes için aynıdır. NPM veya Yarn workspaces kullanılarak, core paketine bağımlılık şu şekilde tanımlanabilir:

```
"dependencies": {  
  "@org/core-business": "workspace:*"  
}
```

veya yine görece `file:` path'leri kullanılabilir; ancak bu kez tüm geliştiricilerde klasör yapısı sabit olduğu için path çözümleri tutarlı kalır. Monorepo yaklaşımının avantajları şunlardır:

- Tek bir Git deposu ve tek bir `node_modules` hiyerarşisi ile geliştirme ortamı sadeleşir.
- Core ve istemci projeleri arasında değişiklik yapmak, tek commit akışı içinde yönetilebilir.
- IDE ve araçlar, projeler arası geçişi daha sorunsuz destekler.

Core Katmanının Temel İlkesi

Core (business) katmanı, birden fazla istemci (mobil, web, gerekirse sunucu tarafı) tarafından ortaklaşa kullanılan *iş mantığını* içerir. Bu katmanda:

- **Olması gerekenler:**
 - TypeScript tip tanımları,
 - Hesaplama ve karar fonksiyonları,
 - API istemci / servis katmanı,
 - UI'dan bağımsız, saf fonksiyonel veya durum içeren mantık.
- **Olmaması gerekenler:**
 - Herhangi bir *UI kütüphanesine* ait tip veya bileşen (örneğin `View`, `Text`, `Button`, `div` vb.),
 - Platforma özgü global nesneler (örneğin `window`, `document`, `Platform`),
 - React Native'e veya DOM'a doğrudan bağımlı olan kod.

Bu yaklaşım sayesinde core katmanı, mobil ve web istemcileri arasında yeniden kullanılabilir, test edilebilir ve uzun vadede sürdürülebilir bir domain mantığı sunar.

Örnek Senaryolar: Core Katmanında Business Kodunun Kurgulanması

Bu bölümde, core katmanında tanımlanan işlevlerin mobil ve web tarafınca nasıl tüketileceğini *UI koduna girmeden*, soyut ve basit örneklerle ele alıyoruz.

Örnek 1: “Aşk Uyumu” Hesaplayan Business Fonksiyonu

Tip ve Fonksiyon Tanımı (Core Katmanında)

Core projede aşağıdaki gibi bir tip ve fonksiyon tanımlandığını varsayalım:

```
// core-business/src/types.ts
export type PersonProfile = {
  id: string;
  name: string;
  age: number;
  interests: string[]; // ["dance", "travel", "books"]
  city: string;
};

// core-business/src/loveScore.ts
import type { PersonProfile } from './types';

export type LoveScoreResult = {
  score: number; // 0 - 100
  commonInterests: string[];
  distancePenalty: number;
};

export function calculateLoveScore(
  you: PersonProfile,
  partner: PersonProfile,
): LoveScoreResult {
  // ... skor hesaplama algoritması ...
}
```

Bu fonksiyon:

- Sadece tipleri ve saf hesaplama mantığını içerir,
- Herhangi bir UI veya platform bağımlılığı yoktur,
- Hem mobil hem web istemcisi tarafından kullanılabilir durumdadır.

Mobil ve Web Tarafının Kullanım Mantığı

- Mobil istemci (React Native) tarafında:
 - Ekran, kullanıcıdan iki profil ile ilgili verileri alır (form, seçim vb. yoluyla),
 - Bu verilerden `PersonProfile` tipinde iki nesne oluşturulur,
 - Daha sonra core fonksiyonu çağrılır:

```
const result = calculateLoveScore(youProfile, partnerProfile);
```
 - UI katmanı, `result.score` ve `result.commonInterests` gibi alanları ekranda uygun şekilde gösterir.
- Web istemci (React / başka bir framework) tarafında:
 - Benzer şekilde, kullanıcıdan profil bilgileri alınır,
 - Aynı `calculateLoveScore` fonksiyonu çağrılır,
 - Sonuçlar web arayüzünde (örneğin tablo, kart, grafik) gösterilir.

Önemli olan nokta, *her iki istemcinin de aynı business fonksiyonunu çağırması*, yani iş mantığının tek bir yerde (core katmanında) konumlandırılmasıdır.

Örnek 2: Randevu / İlaç Hatırlatma Zamanı Hesaplama

Core Katmanında Zamanlama Mantığı

Core projede, günlük bir hatırlatmanın bir sonraki tetiklenme zamanını hesaplayan basit bir yapı tanımlanabilir:

```
// core-business/src/schedule.ts
export type Reminder = {
  id: string;
  title: string;
  hour: number; // 0-23
  minute: number; // 0-59
};

export function getNextReminderTime(reminder: Reminder, now: Date): Date {
  const next = new Date(now);
  next.setHours(reminder.hour, reminder.minute, 0, 0);

  if (next <= now) {
    next.setDate(next.getDate() + 1);
  }

  return next;
}

export function formatTimeLabel(reminder: Reminder): string {
  const hh = reminder.hour.toString().padStart(2, '0');
  const mm = reminder.minute.toString().padStart(2, '0');
  return `${hh}:${mm}`;
}
```

Bu fonksiyonlar:

- Tarih ve saat hesaplamasını core içinde merkezi olarak tanımlar,
- İstemcilerin yalnızca **Reminder** nesnesi ve **now** parametresi vererek sonuç üretmesini sağlar,
- UI tarafına sadece *etiket* (**formatTimeLabel**) ve *bir sonraki tetiklenme zamanı* (**getNextReminderTime**) gibi sade çıktılar sunar.

Mobil ve Web Tarafında Kullanım Mantığı

- Mobil uygulama:
 - Kullanıcıya hatırlatmanın başlığını ve saatini gösterir,
 - Arka planda:

```
const next = getNextReminderTime(reminder, new Date());  
const label = formatTimeLabel(reminder);
```

- `label` değerini UI’da gösterir,
 - `next` değeri, sistem bildirimleri veya planlama için kullanılır.
- Web uygulaması:
 - Örneğin bir yönetim panelinde aynı **Reminder** listesini tablo halinde gösterebilir,
 - Aynı fonksiyonları kullanarak, her satırda hem insan-okur biçiminde zamanı hem de bir sonraki tetiklenme tarihini gösterebilir.

Örnek 3: Core Katmanında Durum (State) Mantığı

Core katmanında, UI bağımlılığı olmadan, basit durum yönetimi mantığını içeren bir sayaç da tanımlanabilir. Örneğin saf TypeScript fonksiyonları ile:

```
// core-business/src/counter.ts
export type CounterState = {
  value: number;
};

export function createCounter(initial: number = 0): CounterState {
  return { value: initial };
}

export function increment(state: CounterState): CounterState {
  return { value: state.value + 1 };
}

export function decrement(state: CounterState): CounterState {
  return { value: state.value - 1 };
}

export function reset(state: CounterState, initial: number = 0): CounterState {
  return { value: initial };
}
```

Bu yaklaşımda:

- Core yalnızca *durumun nasıl evrileceğini* tanımlar,
- Hangi UI bileşeninin bu durumu yönettiği ve nasıl gösterdiği tamamen istemci projeye bırakılır,
- İstemciler bu fonksiyonları, kendi framework'leri içindeki state yönetim mekanizmalarına entegre ederler.

Mimari Yol Haritası

Bu bölüm, core-mobile-web yapısının zaman içinde nasıl evrilebileceğini fazlara ayrılmış bir yol haritası şeklinde özetler.

Faz 1: Çalışan İlk Sürümü Ortaya Çıkarma

İlk aşamada öncelik, *çalışan bir ürün* ortaya çıkarmaktır. Bu fazda:

- Mobil uygulama (React Native) UI ve temel ekran mantığı ile geliştirilir,
- Basit bir arka uç (REST, GraphQL vb.) kurulur ve istekler doğrudan ekranlardan da çağrılabilir (makul sınırlar dahilinde),
- Yine de mobil projede, en azından:

```
src/  
  screens/  
  components/  
  services/  // api client vb.  
  domain/    // saf business fonksiyonları için basit bir alan
```

gibi kaba bir ayırım yapılması, ileride refactor sürecini kolaylaştırır.

Bu fazda DDD kalıpları zorunlu değildir; amaç, UI ile domain mantığını en azından kısmen ayırmaya başlamaktır.

Faz 2: Business Mantığını Netleştirme

Ürün temel olarak çalışır hale geldikten sonra, aynı iş mantığının farklı ekranlarda tekrarlandığı ve UI'dan ayrıştırılmasının anlamlı olduğu yerler ortaya çıkar. Bu aşamada:

- Mobil projede, `src/domain/` altında saf fonksiyonlar toplanır:
 - `loveScore.ts`,
 - `reminderSchedule.ts`,
 - `questionRules.ts` vb.
- Ekranlar, bu fonksiyonları çağırarak ve sonucu gösteren daha ince katmanlara dönüşür,
- Arka uçtaki iş kuralları da benzer şekilde ayrıştırılmaya başlanabilir.

Bu faz, core projenin doğrudan oluşturulmasından önce bir *hazırlık* niteliği taşır.

Faz 3: Core Projenin Ayrılması ve Web İstemcisinin Eklenmesi

Web istemcisi ihtiyacı ortaya çıktığında, business kodunun ayrı bir core pakete taşınması doğal bir adım haline gelir. Bu fazda:

1. **Core projesi** ayrı bir paket olarak oluşturulur (örneğin `core-business`):
 - İlk aşamada, mobil projedeki `src/domain` altındaki saf fonksiyonlar bu core pakete taşınır,
 - Paket, TypeScript kütüphanesi olarak yapılandırılır (derleme, tip dosyaları, `main` ve `types` alanları).
2. Mobil proje:
 - Önceden yerel `src/domain` içinden import ettiği fonksiyonları, artık `@org/core-business` paketinden ithal etmeye başlar.
3. Web projesi:
 - Yeni bir web istemcisi (React, Next.js, Vite vb.) oluşturulur,
 - Aynı core paketini bağımlılık olarak ekler,
 - UI katmanı, core içindeki business fonksiyonlarını kullanarak sayfaları ve bileşenleri oluşturur.

Bu fazın sonunda:

- Bir core (business) projesi,
- Bir mobil istemci,
- Bir web istemci

olmak üzere üç yapı, net bir rol dağılımıyla birlikte çalışır.

Faz 4: DDD'ye Evrim ve Domain Bazlı Organizasyon

Core projenin içinde domain çeşitliliği arttıkça, dosyaların tek bir düz listede tutulması zorlaşır. Bu aşamada, domain odaklı (DDD benzeri) bir yapılandırmaya geçmek anlamlı hale gelir. Örneğin:

```
core-business/  
  src/  
    dating/  
      domain/  
        entities/  
        services/  
      application/  
      use-cases/  
    reminders/  
      domain/  
      application/  
    subscriptions/  
      domain/  
      application/  
    shared/  
    utils/
```

Bu yapıda:

- Her domain (örneğin **dating**, **reminders**, **subscriptions**) kendi bounded context'i olarak ele alınır,
- **domain** klasörleri, entity, value object ve domain servislerini içerir,
- **application** klasörleri, use-case odaklı senaryoları (*komut* ve *sorgu* mantığını) barındırır,
- İstemci projeler (mobil ve web), doğrudan domain sınıflarına müdahale etmek yerine, **application** katmanındaki use-case'leri çağırır.

Bu faz ile birlikte:

- Core proje, gerçek anlamda domain merkezli bir yapıya kavuşur,
- Yeni özellik eklemek, mevcut bir domain'e use-case ekleme veya yeni bir domain oluşturma şeklinde sistematik hale gelir,
- Mobil ve web istemcileri, yalnızca UI ve kullanıcı etkileşimi katmanına odaklanır.

Sonuç

Çok katmanlı core–mobile–web yaklaşımı, modern istemci tarafı mimarilerinde iş mantığının tek bir merkezde toplanmasını, buna karşılık kullanıcı arayüzü ve platforma özgü ayrıntıların istemci projelere bırakılmasını sağlar. Core katmanının; tip tanımları, hesaplama ve karar fonksiyonları, API istemcileri ve UI'dan bağımsız mantık ile sınırlandırılması, bu katmanın hem mobil hem web hem de gerektiğinde sunucu tarafı süreçler tarafından yeniden kullanılabilir, test edilebilir ve sürdürülebilir olmasını mümkün kılar.

Farklı projeler ve makineler arasında core paketinin paylaşımı için birden fazla yöntem bulunmaktadır. Tek geliştiricili veya tek makine odaklı senaryolarda yerel `file:` bağımlılığı pratik bir çözüm sunarken, çok geliştiricili ve uzun ömürlü projelerde private registry veya Git üstünden paketleme yaklaşımları daha sağlam ve ölçeklenebilir bir temel oluşturur. Monorepo ve workspace kullanımı ise özellikle aynı depo içinde core, mobil ve web projelerini bir arada yönetmek isteyen ekipler için operasyonel sadelik ve tutarlılık sağlar.

Mimari yol haritası, önce çalışan bir ürün ortaya koymayı, ardından iş mantığını UI'dan kademeli olarak ayırmayı, ihtiyaç doğduğunda core projesini ayrı bir paket haline getirmeyi ve domain çeşitliliği arttıkça bu core'u domain odaklı (DDD benzeri) bir yapıya evriltmeyi önerir. Bu adımlı yaklaşım, hem erken aşamada aşırı soyutlama maliyetinden kaçınmaya, hem de proje olgunlaştıkça güçlü, yeniden kullanılabilir ve bakımı kolay bir domain katmanı inşa etmeye imkân verir. Böylece, uzun vadede hem mobil hem web istemciler, tutarlı ve merkezi bir iş mantığı üzerinde yükselen, daha öngörülebilir ve genişlemeye açık bir mimari yapıdan faydalanmış olur.